

Design of an efficient Communication Architecture for Cyber-Physical Production Systems

Florian Kaestner¹, Dirk Kuschnerus², Christoph Spiegel³, Benedikt Janssen⁴ and Michael Huebner⁵

Abstract—Current industrial plants are typically equipped with Programmable Logic Controllers (PLCs) in fixed hierarchical structures. A key paradigm in Industrial Internet of Things (IIoT) is the design of distributed smart embedded systems replacing PLCs and extending their capability to enable energy-efficient, scalable systems offering performance and self-adaptivity to changing economical, environmental and process conditions. To be applicable in industrial contexts, these Cyber-Physical Production Systems (CPPS) have to satisfy strict real-time requirements while providing cost-efficiency and user-friendliness. In addition, the intellectual property of the system stakeholders like business logic and control algorithms must be protected against manipulation and theft within the distributed system. In this paper, we analyze communication approaches and provide a CPPS architecture fulfilling these requirements.

I. INTRODUCTION

Cyber-Physical Systems (CPS) integrate computation and physical processes [1]. The application of CPS for industrial and automation purposes is part of the IIoT strategy and often referred to as Cyber-Physical Production Systems (CPPS). The application of the CPS paradigm in industrial automation is complicated by a combination of requirements and constraints.

Current industrial plants are typically driven by Programmable Logic Controllers (PLCs) with fixed hierarchical structures. The hierarchical structure can be characterized with the Computer Integrated Manufacturing (CIM) reference pyramid model as described in [2]. Within this static concept, control propagates typically downwards and information upwards through the layers in bundles. On the other hand, due to the closeness to the process, data acquisition and control states achieve a better accuracy, due to the closeness to the process, when moving downwards. Although this model works well at the moment, increasing performance capabilities and decreasing costs of embedded systems provide opportunities to flatten or even to disband this hierarchy in the near future. Decentralized automation using smart distributed embedded devices is the best solution for this challenge. The presented concept aims to increase the flexibility, modularity and adaptivity by introducing

methods like self-optimization, self-configuration and self-diagnosis. Furthermore decentralized decision making and transparency of information are key factors regarding these targets.

With the use of CPS a higher level of interoperability and interconnection can be achieved. In the area of industrial decentralized automation this approach is known as vertical integration. CPS, as a participant of the value chain, communicates and collaborates with other CPS and humans to form an intelligent network with autonomous subsystems. Furthermore, industrial providers of automation technology aspire to evaluate process relevant information with the help of new algorithms like machine learning or advanced process controls local to the process. Due to the developing performance of embedded devices and decreasing price this goal is going to be achievable. To further accomplish this goal in a cost- and integration-effective manner it is beneficial to share resources of various embedded platforms. Using such an approach, different providers of industrial technology could include their IP on a single platform. However, this technique requires a certain level of isolation and security, which could be achieved when different institutes include their advanced algorithms on a single platform using appropriate security measures. This results in the effective sharing of quantitatively and qualitatively well acquired information. However, the rearrangement of industrial automation structures has to be done stepwise, which leads to the question of which levels of the CIM pyramid are integrated into a CPPS and which hierarchical barriers this concept is able to break through with current embedded processing and communication technologies.

In this paper we focus on the development of a generic distributed embedded system in order to satisfy the paradigms arising from the CPPS notion as well as workability and practicality considerations regarding industrial needs. This approach takes both new and industrial proven communication technologies into account. Currently no processing or communication technology is able to satisfy all requirements. This is the result of combining different levels of automation and different needs like Quality of Service (QoS), real-time requirements, performance or energy and cost efficiency. Therefore we divide the CPPS integration into a vertical, a horizontal and a sensor/actor communication. Additionally, we propose a design methodology which provides the possibility to share resources of embedded platforms in a secure and uncomplicated manner. As a result, we present a design of a CPPS architecture for decentralized automation suitable

¹Florian Kaestner is with Chair of Embedded Systems, Ruhr-University Bochum, Germany florian.kaestner@rub.de

²Dirk Kuschnerus is with KROHNE Innovation GmbH, Bochum, Germany D.kuschnerus@krohne.de

³Christoph Spiegel is with KROHNE Innovation GmbH, Bochum, Germany C.spiegel@krohne.de

⁴Benedikt Janssen is with Chair of Embedded Systems, Ruhr-University Bochum, Germany benedikt.janssen@rub.de

⁵Michael Huebner is with Chair of Embedded Systems, Ruhr-University Bochum, Germany michael.huebner@rub.de

to many real industrial applications and sensor-actuator interactions.

The remainder of this paper is arranged as follows: Section II examines related work - Section III presents communication concepts for vertical, horizontal and sensor/actor integration - Section IV describes our approach for providing isolation of intellectual property - Section V concludes this paper with an overview of our proposed CPPS architecture.

II. RELATED WORK

The upcoming challenges to implement smart distributed systems as CPPS are key concerns of several studies and concepts. In [5] the authors identify the issues regarding the design of process controls through a heterogeneous network architecture of sensors actuators and processing elements. Due to this scenario sensing, processing, communicating and actuating is done by different elements of the network. This diversity leads to unresolved problems providing heterogeneous communication and processing capabilities. Other issues regarding combinations of real-time operations and cross-layer are addressed as well. To tackle these issues an abstract heterogeneous network consisting of different components with varying processing and communication capabilities is proposed.

In [6] real-time properties for design of heterogeneous industrial automation systems are addressed in detail. Due to this approach the authors introduce a generic terminology characterizing real-time systems independently from their implementation by using a proposed black-box end-to-end flow. The target of their approach is to bundle different end-to-end flows into homogeneous blocks by their criticality characterization.

Model-based CPPS architectures and technologies are outlined in [9]. More precisely the authors focus on the vertical integration to exchange data in customizable formats with remote clients using specific domain models. This work includes an approach to avoid cross-layer communication by model driven engineering.

In reference [7] the current state of CPPS implementations are summarized and analyzed. The authors identify the lack of standardization regarding communication protocols, connectors and physical interfaces, data exchange and management system causing the problems which occurred during CPPS implementations in field tests. As a suitable solution a decomposition of the automation pyramid into several hybrid distributed system is suggested. The specification of the CPPS is given by their levels of operation and communication is managed through dedicated gateways. Therefore, it is not necessary to consider several standardizations.

In [10] the authors introduce software defined networking (SDN), which separates network control logic from the underlying physical routers and switches, for industrial automation. For this purpose Profinet as a standard Ethernet fieldbus is used to separate control and data plane. Furthermore, SDN is applied to Profinet leading to the SDNProfinet approach. In this work practical concept networking management is

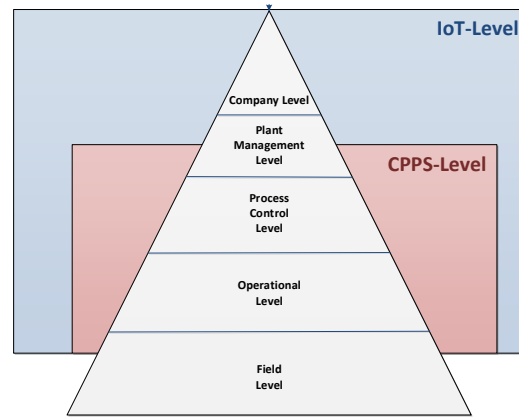


Fig. 1. CIM reference model and CPPS integration

simplified by unifying different network devices and services are delivered using SDN controller applications. Real-time requirements, adaptivity possibilities as well as service oriented aspects are considered to the communication design in this scenario.

Although Industry 4.0 and CPS had become part of the high-tech strategy of the German government in 2011, the idea to break centralized automation into distributed smart systems, accessible via the internet, was also a part of the e-Automation concept described in [11]. This approach is based on a multi agent structure, including software and hardware architecture concepts with a high abstraction level.

III. CPPS INTEGRATION

As described in section I and II, in order to apply the concept of CPPS to real industrial automation tasks several problems have to be addressed. On the one hand, standardization of protocols, software and interfaces is a useful method to ease the implementation and the interaction of devices and networks as mentioned in [7]. On the other hand, as seen in [5] heterogeneous networking elements with dedicated tasks regarding different requirements in different areas of an industrial automation plant are more appropriate to the concept of CPPS. While generally, these approaches are not contrary, a process has to be found to merge these concepts. Furthermore, an appropriate and generic CPPS design including industrial proven communication standards, can contribute to a better implementation success. In section I we mentioned the CIM reference pyramid as an established model to describe the automation hierarchy. The functional separation in 5 different layers breaks the complex task into automation and IT areas. While automation tasks are coupled to real-time constraints, IT tasks usually deal with organizational issues and data mining. Controlling, decentralized decision making and monitoring both areas are involved designing a CPPS. Figure 1 shows the placement of a CPPS and the corresponding IoT segment. This term is based on the integration of current technologies. In our opinion a CPPS should include all tasks regarding the Process Control and

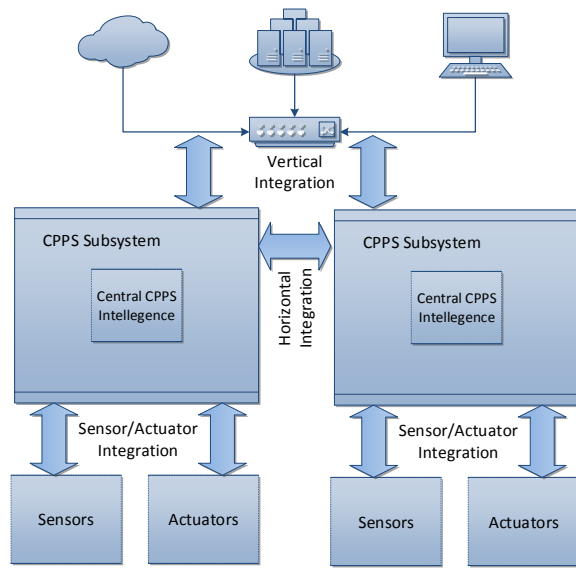


Fig. 2. Conceptual design of a CPPS subsystem

Operational level. Additionally, in order to provide the intelligence achieving conceptual goals like decentralized decision making or controller adaption we suggest to implement some tasks located in the plant management level. These tasks also offer an interface towards upper levels represented by Enterprise Resource Planning (ERP) and Manufacturing Execution Systems (MES). Due to that fact, in this work we disassociate the IoT segment, which includes all levels, from the CPPS area as shown in Figure 1 and focus solely on the conceptual design of a CPPS subsystem. We define a CPPS subsystem as a network of sensors and actuators with an embedded intelligence controlling and monitoring a local process. The size of this network depends on the process. A key factor to determine which sensors and actuators are combined in a CPPS are local arrangements as seen in traditionally distributed systems. Nevertheless, to extend the capabilities of this subsystems and to flatten the hierarchy of the automation pyramid in many ways the CPPS has to be more efficient than traditional PLC subsystems. On the one hand recent technologies, researches and developments are able to overcome some of the difficulties to achieve Industry 4.0 goals with the help of new algorithms and more efficient embedded processing platforms. On the other hand an efficient communication architecture is necessary to combine different requirements from different levels. Due to the fact that no universal communication technology protocol with a corresponding protocol is able to satisfy all demands, we suggest to break the integration of a subsystem into vertical, horizontal and sensor/actuator integration. As a result, we analyze different communication technologies and protocols, which are suitable to the CPPS paradigm. Obviously, a central processing platform within a CPPS subsystem is essential to realize this concept. Figure 2 shows the conceptual design of a CPPS subsystem.

A. Vertical Integration

The extended information exchange requirement between the automation and the IT area throughout all levels have led to development of the OPC Unified Architecture (UA) standard [12]. OPC UA assures interoperability using a platform independent server-client architecture based on standard web technologies and thus provides security as well as information models. Due to these advantages OPC UA has become the de facto standard of vertical integration in industrial automation. The information models are object-oriented and define the organization of data within the address space of the OPC UA server [13], which in this case is the Central CPPS Intelligence. An interesting feature provided by OPC UA is a scalable stack, which is highly important for an embedded device implementation with limited resources. Another popular already standardized IoT protocol is the Message Queue Telemetry Transport (MQTT) protocol [17]. Both of these protocols are based on Transport Layer Protocol / Internet Protocol (TCP/IP). While OPC UA works with a traditional client-server architecture MQTT uses a publisher-subscriber principle. Thus, a message broker is needed distributing messages to clients. Previously, clients have to register their interest to the broker due to their topic. Also published messages have to belong to a specific topic. Therefore, the messages are distributed to only relevant clients with the option of 3 types of quality of service. While MQTT allow a very easy communication with short development cycles and a small footprint, OPC UA is a sophisticated protocol with many opportunities to create advanced information and security models. Each of both IoT protocols has benefits and disadvantages depending on the application requirements. Thus we propose to include both protocols into our CPPS subsystem as described in section IV. For detailed information about the comparison between these protocols we refer to [19]. As information regarding OPC UA [14][16][15] and MQTT [20][21] is abundant due to the standardization and many published research projects, we will not elaborate further on vertical integration, but focus on the horizontal and sensor/actuator integration.

B. Horizontal Integration

While most of the work regarding CPPS communication concentrates on vertical integration, the bridge to the automation area and its needs is often neglected in terms of decentralized automation. These demands are mainly real-time constraints, but also practicability and simply availability of sensors or actuators. In a traditional hierarchical automation the question which fieldbus should be deployed to solve the given problem in an effective manner is typically restricted to the used PLC. Many PLC manufacturers provide their own fieldbuses with little interconnection capabilities to other fieldbuses like Profinet (Siemens) or Ethercat (Beckhoff). The interface between different subsystems with different fieldbuses is usually driven by a gateway. However, without the need to use a PLC we can choose among different fieldbus systems best suited to our requirements of a CPPS architecture with a

wide field of applications. Most of the modern fieldbus systems are based on Ethernet technology except Controller Area Network (CAN).

In case of the horizontal intergration we propose the use of a fieldbus protocol based on Ethernet technology. The main reason for this decision is the high data transmission rate and the real-time capabilities that are possible using this technology. We assume that using decentralized automation with the help of many distributed smart subsystems increases the communication overhead between those subsystems. Although information may be cumulated to higher levels of information abstraction with the help of machine learning or fusion algorithms, the number of subsystems and their tasks and thus the size of the network and the need of excessive information exchange to form distributed intelligence increases the amount of data transfers. Another reason is the substitution of traditional fieldbus systems with Ethernet based fieldbus systems in industry, successively. The most popular fieldbus protocols based on the Ethernet technology are Ethercat, EtherNet/IP, Powerlink, Modbus/TCP, Sercos3 and Profinet [23]. For both, horizontal and sensor-actuator integration, the CPPS architecture has to satisfy hard real-time constraints. In some application scenarios real-time constraints regarding horizontal integration are not given. However, whilst designing a CPPS architecture with a wide field of applications, these requirements should be considered. Unfortunately in computer science, real-time is a term with a wide range of interpretations especially in relation to industrial fieldbus systems. Thus, every mentioned fieldbus owns real-time capabilities defined by its specification. Due to this, a closer look to the arbitration procedures of each fieldbus protocol is unavoidable. In this paper, we define that a fieldbus system possesses hard real-time capabilities if cycle-times smaller than one millisecond are possible and if the arbitration procedures are deterministic allowing an exact calculation of the cycle-time. Thus, fieldbus systems using stochastic arbitration procedures like EtherNet/IP or Modbus/TCP are removed from the selection list. Furthermore, another important requirement is based on the market share. A fieldbus system rarely used in industry should not be considered due to the fact that industrial providers prefer to use proven technology with a wide range of applications and costumers. This leads to the removal of Sercos3, which owns by far the lowest market share of the remaining fieldbus systems. The remaining fieldbus systems are Profinet [26], Ethercat [25] and Powerlink [24]. In contrast to EtherCAT and Profinet, the Powerlink protocol needs no additional hardware support to the Ethernet standard. This protocol uses a mixture of polling and time multiplexing to achieve a deterministic communication. The communication procedure is separated into three cycles: start, isochronous and asynchronous data exchange, which represents the timeslot functionality. Furthermore, the isochronous data exchange is managed with a polling mechanism. Therefore, a Managing Node sends Poll Requests to a specific Node in the network. Due to the broadcast mechanism every Node receives this

message, but only the intended receiver replies. Typically hubs are used for interconnection between these Nodes. Profinet Isonchronous Real Time (IRT), realizing server-client architecture, uses only time multiplexing based on modified hardware switches to comply with hard real-time constraints. A completely different method was applied to realize EtherCAT. This fieldbus is based on the summation frame method exploiting full-duplex capability. Within this method every EtherCAT frame passes all nodes in the network in a sequence in both directions. Therefore, the logical topology of an EtherCAT network is always a ring topology. In order to realize a high throughput EtherCAT slaves read out and write data on the fly with the help of special EtherCAT Slave Controllers (ESC). Thus, the hardware acceleration is located on every Slave Node in the network. Each of these fieldbus system has benefits and disadvantages based on their characteristics and operational areas, thus complicating a fair comparison. Due to a stepwise and easy integration of a CPPS architecture, other important factors that have to be considered are the difficulty of integration, engineering efforts, costs of hardware integration and the accessibility of toolchains supporting the implementation. Within these aspects Profinet is at the lower end of the scale. To the best of our knowledge no open source master implementation is available. Thus, a PLC from Siemens is needed to build a network with this technology. In addition, changing the size of the network entails a huge effort to configure and adjust the network with Profinet IRT due to the time multiplexing nature. Furthermore, special hardware switches are expensive and own low connector capacities and the need of hubs and switches leads to disadvantages realizing line or ring topologies, which are the most popular in automation area. On the basis of this characteristics, we not consider Profinet as a horizontal integration architecture, although critical characteristics are satisfied. The performance capabilities of EtherCAT and Powerlink depend on the size of the network, using topology, performance of the processing hardware and the size of the communication payload.

Table I shows a performance comparison between EtherCAT and Powerlink in a line topology. This rough estimates delivers an idea of how both fieldbus systems work in the most commonly used topology in automation. Although Powerlink has a higher performance capability using all other topologies, EtherCAT benefits from its operating principle in these scenarios. In case of a star topology the estimated cycle time of Powerlink communication in the scenario at the lower end of Table I (200 slaves, 64 Byte payload) is 3257 μ s. However, we suggest using EtherCAT as a horizontal integration architecture due to the advantages of a line topology and the broad and cost efficient availability of ESCs, software stacks and toolchains. In addition, EtherCAT allows us to combine horizontal and vertical integration by leveraging its ability to tunnel Ethernet protocols like TCP/IP. Transferring both OPC UA and horizontal subsystem communication in one communication system drastically adds to the cost- and

energy efficiency of our architecture.

C. Sensor/Actuator Integration

The ability to migrate control logic from a central PLC to decentralized subsystems enables a system architecture with local clusters of sensors and actuators. These local nodes are mostly deeply embedded systems with severe energy restrictions which impedes the provision of EtherCAT communication. Furthermore, our new system architecture allows the local processing and control of these clusters, removing the necessity of globally available sensor data. Therefore, we use CANopen [22], an application layer protocol on top of the CAN bus, for sensor and actuator integration on subsystem level. CANopen allows the installation of compact, energy- and cost-efficient networks while providing priority-based arbitration enabling the design of reliable controllers. The decision to use CANopen was additionally driven by the available application profiles standardizing device data from different domains. As with the intersection between horizontal and vertical communication, EtherCAT serves as central communication gateway also for sensor and actuator integration by allowing the native reuse of the CANopen application profiles. This contributes to the flexibility of our architecture, enabling the shifting of local sensors and actuators into the globally available horizontal subsystem communication with a minimum amount of reconfiguration.

TABLE I
ETHERCAT VS POWERLINK CYCLE TIME ESTIMATIONS IN A LINE
TOPOLOGY

	Powerlink	EtherCAT
Payload frame-size	Ethernet 2 Frame (IEEE 802.3), in between 48 and 1500 Byte without header (18 Byte)	Ethernet 2 Frame (IEEE 802.3), in between 34 and 1488 Byte without header (30 Byte)
approximated delays	0,5 μ s hub delay frame passing (d_{hub}), 45 μ s cycle time starting delay d_{start} , 1 μ s response time master d_{rm} , 8 μ s response time slave d_{rs} , bus delay per Frame d_{bus} depended on payload per Frame ($d_{bus} = (\text{payload} + \text{header}) / 100\text{MBit/s}$)	approximated 2 μ s ESC delay frame passing d_{ESC} , 0 μ s cycle time starting delay d_{start} 50 μ s response time master d_{rm} , 0 μ s response time slave d_{rs} , bus delay per Frame d_{bus} depended on payload per Frame ($d_{bus} = (\text{payload} + \text{header}) / 100\text{MBit/s}$)
cycle time estimate (cte)	50 slaves, 29 Byte payload per slave	
	$d_{bus} = (18 * 8 + 29 * 8) / 100\text{Mbit/s} = 3,76\mu\text{s}$, cte = $d_{start} + 50 * d_{bus} + 50 * d_{rm} + 50 * d_{rs} + 2 * \sum_{i=1}^{49} d_{hub} = 6490\mu\text{s}$	$d_{bus} = (30 * 8 + 1488 * 8) / 100\text{Mbit/s} = 121,44\mu\text{s}$, cte = $d_{start} + 2 * d_{bus} + 50 * d_{ESC} + 2 * d_{rm} = 442\mu\text{s}$
	200 slaves, 64 Byte payload per slave	
	$d_{bus} = (18 * 8 + 64 * 8) / 100\text{Mbit/s} = 6,56\mu\text{s}$, cte = $d_{start} + 200 * d_{bus} + 200 * d_{rm} + 200 * d_{rs} + 2 * \sum_{i=1}^{199} d_{hub} = 23257\mu\text{s}$	$d_{bus} = (30 * 8 + 1488 * 8) / 100\text{Mbit/s} = 121,44\mu\text{s}$, cte = $d_{start} + 18 * d_{bus} + 9 * 200 * d_{ESC} + 18 * d_{rm} = 6685\mu\text{s}$

IV. PROCESS ISOLATION AND INTER-PROCESS COMMUNICATION

Designing a CPPS architecture also involves creating subsystems which exploit the existing resources effectively. Therefore, in a CPPS environment, different vendors of IP should be able to integrate their algorithms on a high performance embedded device in an easy and secure way by reuse of existing hardware and data locally close to the automation process which is mandatory for real-time applications. In order to achieve this target we have to create a CPPS subsystem considering process isolation and interprocess communication. For this purpose we use a hardened embedded linux operating system developed with the help of Yocto [27]. This open source collection project provides templates, tools and techniques to build custom Linux operating systems for various embedded hardware platforms. In order to deal with real-time constraints we have to include a real-time kernel patch (PREEMPT-RT). Process isolation is implemented with the help of Linux containers, which is a virtualization method based on namespaces and c-groups. This method allows running multiple applications isolated in unprivileged containers using a single Linux kernel. For this purpose, LXC offer a wide range of Application Programming Interfaces (APIs) to support configuring the container depended on the need and authorization of a specific application. Thus, only specific areas of the Linux file tree can be mount into the virtualized environment of the container, permissions can be configured, access to the peripheral resources can be limited and network configurations can be set. Due to this configurations each application is described by a recipe in Yocto. This method facilitate dissolving dependencies between the application, system libraries, networking, booting and peripheral access. Next, a light-weighted, scalable and fast inter-process communication is needed in order to connect all isolated containers. Different methods and protocols are available for this purpose like file system- or TCP/IP-based communication. We propose to use TCP/IP based communication technologies, because these are typically less susceptible to bottlenecks and insecure back doors and own a better scalability. Therefore, we connect each application to one internal network bridge. MQTT, introduced in section III-A, using a publisher/subscriber communication technique is perfectly suitable as an inter-process communication between applications isolated in separate LXC containers. A central MQTT topic list is used to support developers implementing applications on this CPPS subsystem. Within this list, application programmers are able to search for topics of interest or offer topics to share with other applications. As a result the topic list represent the link between all containers. All containers are equipped with a MQTT client running in a separate thread and a joint data storage acting as a shared memory protected by semaphores. The central MQTT broker is implemented as a single isolated application. To realize vertical integration we propose to use either MQTT or OPC UA. Within the configuration of the system we are able to

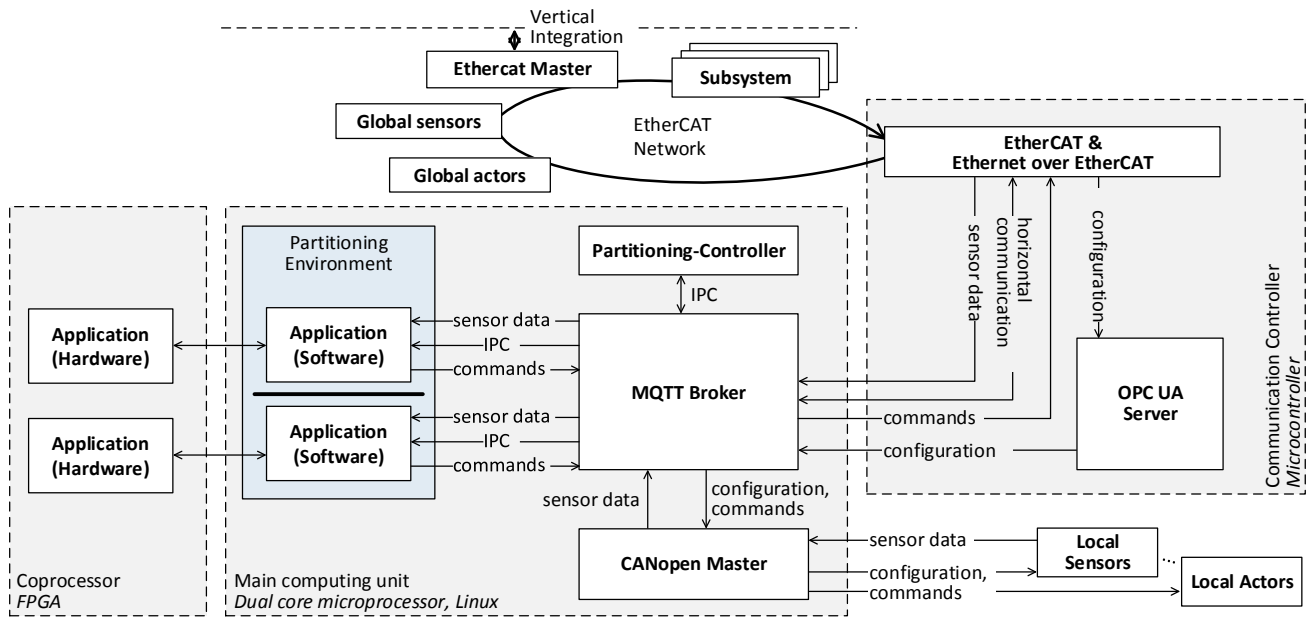


Fig. 3. Proposed CPPS system architecture

allow certain containers access to the Ethernet stack with the help of port forwarding. Thus, we can switch between the container including the MQTT Broker or the OPC UA server interacting with the higher level automation levels. *Systemd* is used as an init system in order to boot each component and application in a specific sequence to avoid dependency issues. All scripts and isolated applications, described with recipes, are summed up to form a custom Yocto layer. As a result, this methodology represents an easy and scalable method to deal with process isolation on a single platform in a secure and efficient manner.

V. CONCLUSION

In this paper, we introduce a communication architecture for CPPS, tackling the challenges of horizontal, vertical and sensor/actor integration as well as partitioning of the intellectual property of various stakeholders. Figure 3 gives an overview of the system architecture. The upper part of the figure shows the EtherCAT network described in section III horizontally connecting subsystems, global sensors and actuators and providing a vertical communication path to company networks via the EtherCAT master. Each subsystem integrates applications and communication using a combination of three computing resources. The communication controller is a microcontroller processing the tunneling of TCP/IP traffic through the EtherCAT network and providing the OPC UA server for subsystem configuration. The partitioning environment for protected applications is deployed on a dual-core microprocessor running our custom-built Linux distribution. Each application can additionally use hardware acceleration provided by an Field Programmable Gate Array (FPGA) embedded in the coprocessor, enabling

complex control and monitoring tasks. The communication of applications is centrally processed via a MQTT broker to further enforce isolation. The broker enables inter process communication and connects the applications to local CANopen sensors and actuators as well as to the overall system via the communication controller. In this paper we use the combination of communication principles and a partitioning framework based on Linux containers in order to create an architecture, which provides a flexible, decentralized infrastructure for future CPPS. As a result, our design can provide the necessary integration needed in various domains.

VI. ACKNOWLEDGMENT

This research has been supported by the DnSPro project funded by the German Federal Ministry of Education and Research (BMBF).

REFERENCES

- [1] Lee, Edward A., *Cyber Physical Systems: Design Challenges*, EECS Department, University of California, Berkeley, 2008, Jan.
- [2] D.P. Buse and Q.H. Wu, *IP Network-based Multi-agent Systems for Industrial Automation: Information Management, Condition Monitoring and Control of Power Systems*, Springer, 2007, Chapter 1: Introduction.
- [3] U. Mnks and V. Lohweg, *2014 4th International Workshop on Cognitive Information Processing (CIP)*, Fast evidence-based information fusion, 2014, May, ISSN=2327-1671.
- [4] B. Can and C. Heavey and K. E. Kabak, *Proceedings of the Winter Simulation Conference 2014*, Generating operating curves in complex systems using machine learning, 2014, Dec, ISSN=0891-7736.
- [5] Yunbo Wang, Mehmet C. Vuran, Steve Goddard, *ACM SIGBED Review*, vol. 5, no. 1, p. 12, 2008, *Cyber-physical Systems in Industrial Process Control*, 2008.
- [6] S. Grner and U. Epple, *Industrial Electronics Society, IECON 2015 - 41st Annual Conference of the IEEE*, Real-time properties for design of heterogeneous industrial automation systems, 2015, Nov.

- [7] H. Igor and J. Bohuslava and J. Martin, 2016 IEEE 20th Jubilee International Conference on Intelligent Engineering Systems (INES), Proposal of communication standardization of industrial networks in Industry 4.0, 2016, June.
- [8] P. D. Roth, Electricity Distribution, 2001. Part 1: Contributions. CIRED. 16th International Conference and Exhibition on (IEE Conf. Publ No. 482), Communications architecture in modern distribution systems, 2001, ISSN=0537-9989.
- [9] F. Prez and E. Irisarri and D. Orive and M. Marcos and E. Estevez, 2015 IEEE 20th Conference on Emerging Technologies Factory Automation (ETFA), A CPPS Architecture approach for Industry 4.0, 2015, Sept, ISSN=1946-0740.
- [10] K. Ahmed and J. O. Blech and M. A. Gregory and H. Schmidt, Parallel and Distributed Systems (ICPADS), 2015 IEEE 21st International Conference on, Software Defined Networking for Communication and Control of Cyber-Physical Systems, 2015, Dec.
- [11] H. Li and C. Zhong, Intelligent Computation Technology and Automation (ICICTA), 2008 International Conference on, E-Automation and the Development of Substation Automation Architecture, 2008, Oct.
- [12] OPC Foundation, OPC UA Specification, Release 1.03, 2015, <https://opcfoundation.org/developer-tools/specifications-unified-architecture/part-1-overview-and-concepts>.
- [13] A. Fernbach and W. Kastner and S. Mtzler and M. Wollschlaeger, Proceedings of the 2014 IEEE Emerging Technology and Factory Automation (ETFA), An OPC UA information model for cross-domain vertical integration in automation systems, 2014, Sept, ISSN=1946-0740.
- [14] D. Grossmann and K. Bender and B. Danzer, SICE Annual Conference, 2008, OPC UA based Field Device Integration, 2008.
- [15] R. Cupek and A. Maka, Emerging Technologies and Factory Automation (ETFA), 2010 IEEE Conference on, OPC UA for vertical communication in logistic informatics systems, 2010, Sept, ISSN=1946-0740.
- [16] R. Kyusakov and H. Mkitaavola and J. Delsing and J. Eliasson, IECON 2011 - 37th Annual Conference on IEEE Industrial Electronics Society, Efficient XML Interchange in factory automation systems, 2011, Aug, ISSN=1553-572X.
- [17] Oasis, MQTT Specification Version 3.1.1, 2014, Oct, <http://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>.
- [18] Roger A Light, Mosquitto: server and client implementation of the MQTT protocol, The Journal of Open Source Software, 2017, May.
- [19] L. Durkop and B. Czybik and J. Jasperneite, 2015 18th International Conference on Intelligence in Next Generation Networks, Performance evaluation of M2M protocols over cellular networks in a lab environment, 2015, Feb.
- [20] A. Sahadevan and D. Mathew and J. Mookathana and B. A. Jose, 2017 IEEE 4th International Conference on Cyber Security and Cloud Computing (CSCloud), An Offline Online Strategy for IoT Using MQTT, 2017, Jun.
- [21] S. Ahmed and A. Topalov and N. Shakev, 2017 IEEE International Workshop of Electronics, Control, Measurement, Signals and their Application to Mechatronics (ECMSM), A robotized wireless sensor network based on MQTT cloud computing, 2017, May.
- [22] CiA301, CiA ? CAN in Automation, CANopen application layer and communication profile, 2011, Feb, <https://www.can-cia.org/standardization/specifications/>.
- [23] R. Zurawski, The Industrial Communication Technology Handbook, Industrial Information Technology, 2005.
- [24] Ethernet Powerlink Standardization Group (EPNG), Powerlink Communication Profile Specification, 2005, May, <http://www.ethernet-powerlink.org>.
- [25] Ethercat Technology Group (ETG), Ethercat Automation Protocol, 2007, Oct, <https://www.ethercat.org>.
- [26] Profibus and Profinet International, Profinet IO Specification, 2011, Nov, <http://www.profibus.com>.
- [27] Linux Foundation, Yocto Project Overview, 2010, <https://www.yoctoproject.org>.